

PORTADA DE
MNEMOSYNE

AUTOR: MARÍA BUZARHON

05/2026



I.E.S. LAGUNA DE JOATZEL

TRABAJO FIN DE CICLO

MNEMOSYNE

CICLO FORMATIVO DE GRADO SUPERIOR

DESARROLLO DE APLICACIONES WEB

FECHA 05/2026

María Buzarhon Navarro

Tutor: Alain Fernández

ABSTRACT Y PALABRAS CLAVE

El presente proyecto surge como respuesta a la fragmentación de la información característica del ecosistema digital contemporáneo. Actualmente, la planificación y gestión de actividades culturales exige al usuario la navegación constante entre múltiples plataformas independientes, lo que genera una fricción operativa que penaliza la productividad y degrada la experiencia de uso.

Ante este escenario, nace Mnemosyne, una solución orientada a la convergencia de servicios mediante el desarrollo de una interfaz móvil unificada. El proyecto centraliza la consulta de exposiciones, la gestión de noticias culturales y la adquisición de entradas, optimizando el flujo de trabajo del usuario final en un único entorno digital coherente y fluido.

- Descripción Técnica

Arquitectura del Sistema

La solución se ha materializado como una aplicación nativa para dispositivos móviles, desarrollada íntegramente en el entorno Android Studio utilizando el lenguaje Kotlin. Para garantizar un sistema robusto, mantenible y escalable, se ha implementado una arquitectura basada en el patrón MVVM (Model-View-ViewModel), asegurando una separación estricta entre la lógica de negocio (centralizada en repositorios) y la interfaz de usuario.

Backend y Persistencia (BaaS)

En lugar de un backend tradicional, el proyecto adopta un modelo de Backend as a Service (BaaS) mediante la integración del ecosistema Firebase. Esta infraestructura permite gestionar la lógica de servidor de forma eficiente:

- Autenticación: Se utiliza Firebase Auth para el control de acceso y la seguridad de las cuentas de usuario.
- Base de Datos: La persistencia de la información (museos, noticias, carritos y pedidos) se realiza en Cloud Firestore, una base de datos NoSQL que permite la sincronización de datos en tiempo real.
- Lógica de Servidor Segura: Para procesos críticos que requieren un entorno de ejecución seguro, como la pasarela de pagos, se han implementado Firebase Cloud Functions, protegiendo la integridad de las transacciones financieras.

Gestión Multimedia y Rendimiento

Para optimizar el rendimiento y el almacenamiento de la aplicación, el tratamiento de activos multimedia se gestiona de forma híbrida:

- Firebase Storage: Administra las imágenes de perfil de los usuarios asociadas a su UID.
- Cloudinary: Entrega el contenido visual de gran volumen (como las galerías de exposiciones), asegurando una carga progresiva y alta fidelidad sin penalizar el consumo de datos.

Integración y Desarrollo Continuo (CI/CD)

- Control de Versiones con GitHub: Gestión del código fuente mediante flujos de trabajo Git, permitiendo un seguimiento exhaustivo de los cambios en la aplicación y las Cloud Functions.
- Firebase CLI: Herramienta utilizada para el despliegue automatizado de reglas de seguridad y funciones de nube, garantizando la paridad entre el código local y el entorno de producción.

- Conclusión

La arquitectura de Mnemosyne, fundamentada en la combinación de un desarrollo nativo en Android (Kotlin) y el ecosistema Firebase, proporciona una solución robusta con una experiencia de usuario fluida y reactiva. El uso de una infraestructura Serverless asegura que la aplicación sea altamente escalable, eliminando la carga operativa de gestionar servidores físicos y permitiendo una solución moderna, segura y fácilmente mantenible.

Palabras clave: Android Nativo, Kotlin, Firebase, Cloud Functions, MVVM, Ecosistema Cultural.

ABSTRACT Y KEYWORDS

This project was developed as a response to the information fragmentation characteristic of today's digital ecosystem. Currently, planning and managing cultural activities requires users to constantly switch between multiple independent platforms, creating operational friction that hampers productivity and degrades the user experience.

In this context, Mnemosyne was created as a solution focused on service convergence through a unified mobile interface. The project centralizes exhibition inquiries, cultural news management, and ticket acquisition, optimizing the end-user's workflow within a single, coherent, and fluid digital environment.

- Technical Description

System Architecture

The solution was implemented as a native mobile application, developed entirely within Android Studio using Kotlin. To ensure a robust, maintainable, and scalable system, the architecture is based on the MVVM (Model-View-ViewModel) design pattern, ensuring a strict separation between business logic (Repositories) and the user interface.

Infrastructure and Persistence (BaaS)

Instead of a traditional backend, the project adopts a Backend as a Service (BaaS) model by integrating the Firebase ecosystem, allowing for efficient infrastructure management:

- **Authentication:** Identity management and secure access control are handled via Firebase Auth.
- **Database:** Structured data persistence (museums, news, carts, and orders) is managed through Cloud Firestore, ensuring real-time data synchronization.
- **Server-side Logic:** Critical and secure processes, such as the payment gateway integration, are executed using Firebase Cloud Functions.
- **Multimedia Management:** A hybrid approach is used for assets, leveraging Firebase Storage for user profiles and Cloudinary for high-volume visual content, optimizing network performance.

Continuous Integration and Deployment (CI/CD)

- **Version Control:** GitHub was implemented to orchestrate the source code and track changes across the application and server-side function repositories.
- **Automated Deployment:** Workflows were established using the Firebase CLI, ensuring that security rules and cloud functions are always synchronized with the stable version of the code.

Conclusion

The architecture of Mnemosyne, built on native Kotlin development and the power of Firebase, delivers an advanced technological solution with a reactive and fluid user experience. The Serverless approach ensures high scalability and reduces operational overhead, resulting in a robust and maintainable solution that meets the demands of today's cultural sector. Keywords: Native Android, Kotlin, Firebase, Cloud Functions, MVVM, Cultural Ecosystem.

ÍNDICE

SECCIÓN I: RESUMEN DEL PROYECTO.....	8
I. Presentación.....	8
II. Contexto.....	8
III. Objetivos.....	8
SECCIÓN II: DESARROLLO.....	10
IV. Temporalización y ciclo de vida.....	10
Repartición del trabajo:.....	11
V. Trabajo de Análisis.....	12
Descripción general:.....	12
Casos de uso:.....	17
VI. Trabajo de Diseño.....	21
Modelo de datos.....	22
Diseño de módulos/ficheros de código.....	23
Diseño/diagramas de clases.....	25
Diagrama de navegación.....	26
VI. Trabajo de Implementación.....	27
VI.I - Introducción:.....	27
VI.II - Tecnologías y Herramientas.....	27
VI.III - Desarrollo.....	28
VI.IV. Justificación.....	35
VIII - Trabajo de Pruebas.....	37
VIII.I -Introducción.....	37
VIII.II - Pruebas del sistema de autenticación.....	38
Trabajo de Formación.....	43

BIBLIOGRAFÍA.....	44
--------------------------	-----------

SECCIÓN I: RESUMEN DEL PROYECTO

I. Presentación

Mnemosyne es una aplicación móvil nativa para Android que surge del interés personal de su autora por el sector cultural. Su objetivo es hacer que el acceso a la cultura sea tan sencillo e intuitivo como el de cualquier otro servicio digital, reuniendo en un único entorno exposiciones, noticias y merchandising de los museos colaboradores.

II. Contexto

El ecosistema digital actual obliga al usuario interesado en la cultura a dispersarse entre múltiples aplicaciones y páginas web, encontrando en muchos casos información desactualizada o difícil de localizar. Esta fragmentación resulta especialmente frustrante tanto para el público veterano como para quienes dan sus primeros pasos en el mundo del arte y necesitan una experiencia más accesible. Mnemosyne nace como respuesta directa a esta necesidad.

III. Objetivos

El objetivo principal del proyecto es ofrecer una plataforma unificada que elimine la fricción entre el usuario y la cultura. Para ello, la aplicación busca centralizar la consulta de exposiciones activas y próximas, facilitar el acceso a noticias culturales, integrar la compra de entradas y merchandising, y garantizar una experiencia fluida e intuitiva para todo tipo de usuarios.

SECCIÓN II: DESARROLLO

IV. Temporalización y ciclo de vida.

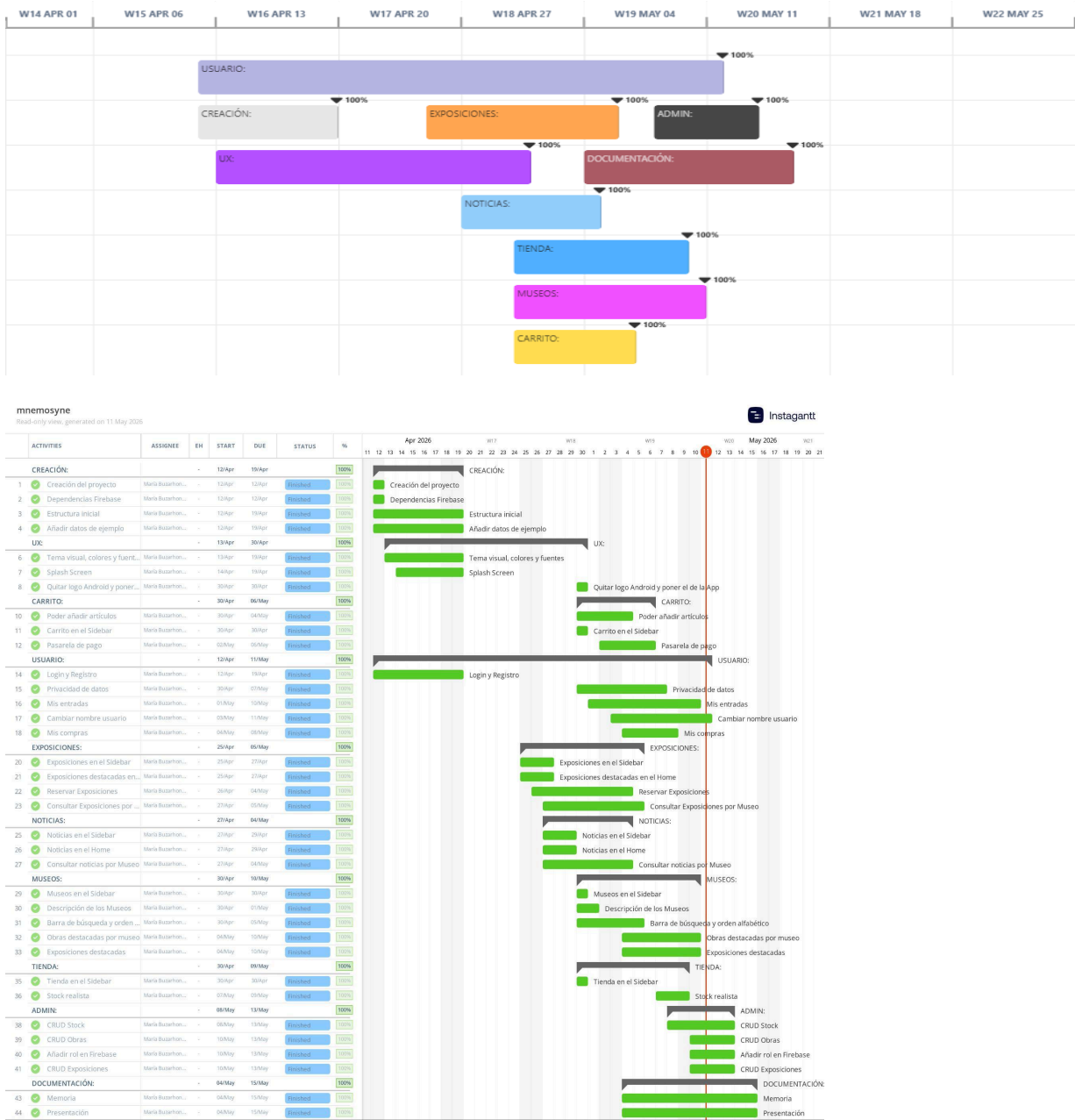


Figura 1 y 2. Diagrama de Gantt. Representación temporal de las actividades del proyecto mnemosyne mediante un diagrama de Gantt, indicando fechas de inicio, finalización y progreso de cada módulo implementado. proyecto “Mnemosyne” con tareas y cronograma entre abril y mayo de 2026.

El proyecto se desarrolló entre abril y mayo de 2026 siguiendo un ciclo de vida iterativo e incremental, avanzando progresivamente por módulos: creación inicial, usuarios, carrito, exposiciones, noticias, museos, tienda, panel de administración y documentación. Todo el desarrollo fue realizado individualmente por María Buzarhon.

V. Trabajo de Análisis.

Descripción general:

En la experiencia de uso pueden diferenciarse dos roles:

1. Usuario:

En la experiencia de uso pueden diferenciarse dos roles principales: Usuario y Administrador.

A continuación se describen en detalle las funcionalidades y el análisis de cada pantalla para cada rol.

1.1 - Pantalla de Inicio:

Funcionalidad: La pantalla de inicio actúa como núcleo central de la aplicación. Al acceder el usuario visualiza un resumen dinámico con las exposiciones y noticias marcadas como destacadas por los administradores de cada museo. Los contenidos se obtienen en tiempo real desde Firestore mediante el HomeRepository, que recorre todos los museos y filtra aquellos documentos cuyo campo destacado es true (verdadero). Desde esta pantalla el usuario puede navegar directamente a las diferentes secciones principales (Exposiciones, Museos, Noticias, Tienda y Carrito) y acceder al menú lateral.

Análisis: El diseño de la pantalla de inicio responde a la necesidad de ofrecer una entrada atractiva y relevante a la aplicación. Mostrar únicamente contenido destacado evita la sobrecarga de información y permite a los administradores controlar qué es lo primero que ve el usuario sin necesidad de actualizar la app.

La carga asíncrona con corrutinas de Kotlin garantiza que la interfaz no se bloquee mientras se obtienen los datos de Firestore.

1.2 - Pantalla de Exposiciones:

Funcionalidad: Esta pantalla muestra el listado completo de exposiciones disponibles en todos los museos. El `ExposicionRepository` recorre la colección de museos en Firestore y obtiene las subcolecciones de exposiciones de cada uno, incluyendo los tipos de entrada.

El usuario puede filtrar exposiciones por museo mediante chips interactivos y acceder al detalle de cada una. En el detalle se muestra la imagen, las fechas de inicio y fin, la descripción completa y los tipos de entrada disponibles, desde donde puede añadir entradas al carrito.

Análisis: La estructura de datos jerárquica en Firestore (museos -> exposiciones) permite una organización natural del contenido y facilita la gestión por parte del administrador. El sistema de tipos de entrada ofrece flexibilidad para que cada exposición defina sus propias categorías de precio, lo que resulta especialmente útil en un contexto museístico real donde los precios varían según el perfil del visitante.

1.3 - Pantalla de Tienda:

Funcionalidad: La tienda muestra los productos físicos disponibles para compra, organizados en una cuadrícula de dos columnas. Los productos se obtienen del `StockRepository`, que recorre todos los museos y agrega el stock de cada uno en una lista unificada. Cada producto se muestra con su imagen, nombre, precio y disponibilidad. Al acceder al detalle el usuario puede ver la información completa y añadir el producto al carrito si hay stock disponible, si no lo hay, el botón se deshabilita automáticamente.

Análisis: La presentación en cuadrícula es adecuada para productos de tienda, ya que permite visualizar más artículos de forma simultánea y facilita la comparación visual. El control de stock en tiempo real desde Firestore evita

situaciones donde un usuario intente comprar un producto agotado, mejorando así la fiabilidad del sistema de compras.

1.4 - Pantalla de Museos:

Funcionalidad: Esta sección presenta el catálogo completo de museos disponibles en la aplicación. Incluye una barra de búsqueda que filtra en tiempo real por nombre y ciudad, y los resultados se presentan ordenados alfabéticamente. El MuseoRepository obtiene los documentos de la colección raíz de museos en Firestore. Cada tarjeta muestra el nombre del museo, la ciudad, un extracto de la descripción y un indicador visual si el museo está marcado como destacado. Al acceder al detalle se muestra la imagen (cargada desde una URL externa vía la librería Coil), la descripción completa y un botón que abre la web oficial en el navegador.

Análisis: La búsqueda y el orden alfabético son funcionalidades clave cuando el catálogo de museos crece. Aplicar el filtro y el ordenamiento en el lado del cliente (sobre la lista ya cargada) es una decisión eficiente para volúmenes de datos moderados como el de esta aplicación, evitando consultas adicionales a Firestore. El uso de Coil para la carga de imágenes desde URLs externas (Cloudinary) permite a los administradores actualizar las imágenes sin necesidad de publicar una nueva versión de la app.

1.5 - Pantalla de Noticias:

Funcionalidad: La pantalla de noticias agrega las publicaciones de todos los museos en un único listado. El NoticiaRepository recorre la subcolección de noticias de cada museo y las combina. El usuario puede filtrar por museo mediante chips y acceder al detalle completo de cada noticia, que incluye imagen, título, museo de origen, fecha y contenido.

Análisis: Centralizar las noticias de múltiples museos en una sola pantalla aumenta el valor de la aplicación como agregador de contenido cultural. El filtro por museo permite a usuarios con interés en un centro concreto acotar

rápidamente la información relevante. Las fechas se normalizan en el repositorio para soportar tanto el tipo Timestamp de Firestore como cadenas de texto, garantizando compatibilidad con datos existentes.

1.6 - Pantalla de Perfil:

Funcionalidad: La pantalla de perfil muestra los datos del usuario autenticado: nombre, correo electrónico, foto de perfil y fecha de registro. El UsuarioRepository obtiene el documento del usuario de la colección usuarios usando el UID de Firebase Auth. Si el documento no existe (usuario recién registrado), se crea automáticamente con los datos de Auth. El usuario puede editar su nombre directamente desde la app y consultar sus entradas adquiridas, que se obtienen del PedidoRepository desde la subcolección pedidos del usuario ordenada por fecha descendente.

Análisis: Mantener los datos del usuario en Firestore (en lugar de depender únicamente de Firebase Auth) aporta flexibilidad para añadir campos adicionales en el futuro como favoritos o preferencias. La creación automática del documento si no existe garantiza que el perfil funcione correctamente incluso si el flujo de registro tuvo algún problema al crear el documento inicial.

1.7 - Pantalla de Carrito y proceso de compra:

Funcionalidad: El carrito persiste en Firestore bajo la subcolección ítems del documento del usuario en la colección carritos. Esto permite que el carrito se mantenga entre sesiones y dispositivos. El CarritoRepository gestiona las operaciones de añadir, eliminar, actualizar cantidad y vaciar el carrito. El proceso de pago está integrado con Stripe. Una vez completado el pago, se registra un pedido en la subcolección pedidos del usuario mediante el PedidoRepository, y el stock de los productos comprados se decrementa en Firestore.

Análisis: La persistencia del carrito en Firestore diferencia la aplicación de soluciones más simples basadas únicamente en memoria local, ofreciendo una experiencia de compra más robusta. El uso de Stripe garantiza un proceso de pago seguro y conforme a los estándares del sector. El decremento de stock tras

la compra cierra el ciclo de venta de forma coherente con el inventario mostrado en la tienda.

2. Administrador:

El rol de administrador se identifica mediante el campo esAdmin del documento del usuario en Firestore. Cuando este campo es true, la aplicación habilita acceso a un panel de administración que no es visible para usuarios estándar.

2.1 - Panel de Administración:

Funcionalidad: El administrador accede a un panel desde el que puede gestionar todos los contenidos de la aplicación a través del AdminRepository. Las operaciones se organizan por entidad:

Gestión de Exposiciones

El administrador puede crear, editar y eliminar exposiciones de cualquier museo. Al crear o editar una exposición puede definir título, descripción, fechas de inicio y fin, URL de imagen, visibilidad pública y si aparece destacada en el inicio. También puede gestionar los tipos de entrada de cada exposición con nombre, descripción y precio.

Gestión de Noticias

Permite publicar noticias asociadas a un museo concreto, con título, contenido, fecha, imagen y opción de marcarla como destacada para que aparezca en la pantalla de inicio. Las noticias existentes pueden editarse o eliminarse.

Gestión de Obras

El administrador puede añadir, editar y eliminar obras del catálogo de cada museo. Cada obra incluye nombre, autor, siglo, cultura, descripción, imagen y un indicador de disponibilidad que permite ocultarla temporalmente sin eliminarla del sistema.

Gestión de Stock

Permite crear y gestionar los productos de la tienda de cada museo: nombre del producto, precio, cantidad disponible e imagen. Las operaciones de creación, actualización y eliminación se reflejan inmediatamente en la tienda visible para los usuarios.

Análisis

La concentración de todas las operaciones de gestión en un único AdminRepository simplifica el mantenimiento del código y establece un punto de control claro para las operaciones de escritura en Firestore. La distinción de roles mediante un campo en el documento del usuario (en lugar de un sistema de autenticación separado) es una solución pragmática y adecuada para el alcance de la aplicación.

El hecho de que el administrador gestione imágenes mediante URLs externas (Cloudinary) elimina la necesidad de Firebase Storage de pago, reduciendo costes operativos sin sacrificar la capacidad de actualizar contenidos visuales de forma dinámica. La arquitectura de Firestore con subcolecciones anidadas bajo cada museo (exposiciones, noticias, obras, stock) facilita la aplicación de reglas de seguridad granulares y permite escalar el sistema añadiendo nuevos museos sin modificar la estructura de datos existente.

Casos de uso:

Actor: Usuario

CU1 – Visualizar pantalla de inicio

Descripción: El usuario accede a la pantalla principal donde se muestran contenidos destacados.

Precondición: Usuario autenticado.

Flujo principal:

- El sistema carga exposiciones y noticias destacadas desde Firestore.

- Se muestran en la pantalla de inicio.
- El usuario puede navegar a otras secciones.

CU2 – Consultar exposiciones

Descripción: El usuario visualiza el listado de exposiciones disponibles.

Flujo principal:

- El sistema obtiene exposiciones de todos los museos.
- El usuario filtra por museo si lo desea.
- El usuario accede al detalle de una exposición.

CU3 – Consultar y comprar entradas

Descripción: El usuario selecciona entradas de una exposición.

Flujo principal:

- El usuario selecciona tipo de entrada.
- Añade al carrito.
- Posteriormente realiza el pago mediante Stripe.

CU4 – Consultar tienda de productos

Descripción: El usuario visualiza productos disponibles en la tienda.

Flujo principal:

- El sistema muestra productos en cuadrícula.
- El usuario accede al detalle de un producto.
- Puede añadirlo al carrito si hay stock.

CU5 – Gestionar carrito

Descripción: El usuario gestiona los productos seleccionados.

Flujo principal:

- Añadir o eliminar productos.
- Modificar cantidades.
- Finalizar compra.

CU6 – Realizar compra

Descripción: El usuario completa el proceso de pago.

Flujo principal:

- El sistema envía la información a Stripe.
- Se confirma el pago.
- Se crea el pedido en Firestore.
- Se actualiza el stock.

CU7 – Consultar museos

Descripción: El usuario explora el catálogo de museos.

Flujo principal:

- El sistema muestra la lista de museos.
- El usuario puede buscar o filtrar.
- Accede al detalle del museo.

CU8 – Consultar noticias

Descripción: El usuario visualiza noticias culturales.

Flujo principal:

- El sistema carga noticias de todos los museos.
- El usuario filtra por museo.
- Accede al detalle de la noticia.

CU9 – Gestionar perfil

Descripción: El usuario visualiza y edita su perfil.

Flujo principal:

- Consulta datos personales.
- Edita nombre.
- Consulta pedidos anteriores.

Actor: Administrador

CU10 – Acceder al panel de administración

Descripción: El administrador accede a funcionalidades avanzadas del sistema.

Precondición: Usuario con campo esAdmin = true.

CU11 – Gestionar exposiciones

Descripción: Crear, editar o eliminar exposiciones.

Flujo principal:

- Crear exposición con datos (título, fechas, imagen, etc.).
- Editar información existente.
- Eliminar exposición.

CU12 – Gestionar noticias

Descripción: Administración de noticias de museos.

Flujo principal:

- Crear noticia.
- Editar noticia.
- Eliminar noticia.
- Marcar como destacada.

CU13 – Gestionar obras

Descripción: Gestión del catálogo de obras.

Flujo principal:

- Añadir obra.
- Editar obra.
- Eliminar obra.
- Marcar disponibilidad.

CU14 – Gestionar stock de tienda

Descripción: Administración de productos de la tienda.

Flujo principal:

- Añadir producto.
- Actualizar stock.
- Eliminar producto.

VI. Trabajo de Diseño

Modelo de datos.

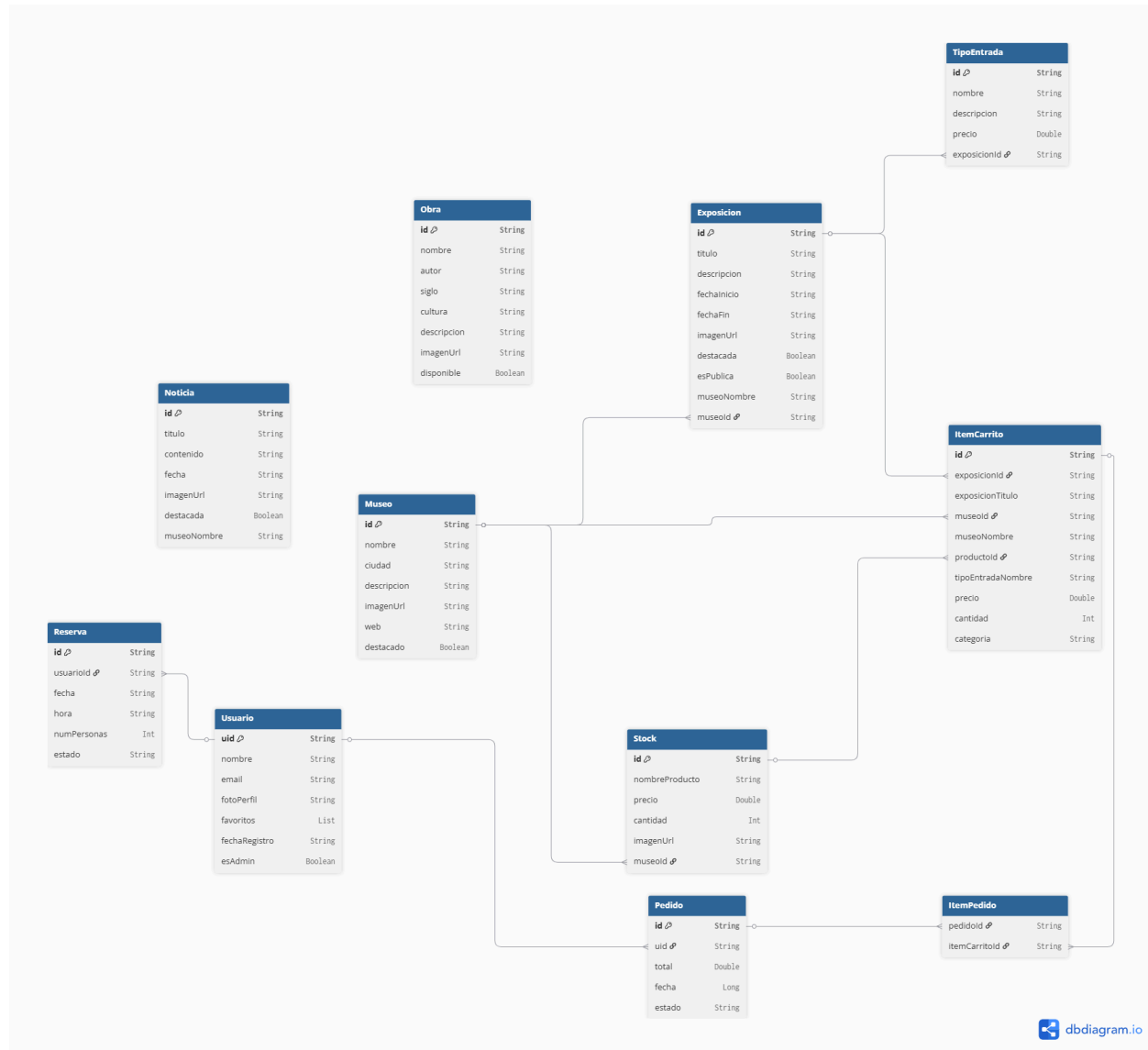


Figura 3. Diagrama de la estructura de datos implementada en Cloud Firestore.

El diagrama muestra las principales entidades de la aplicación y sus relaciones.

Diseño de módulos/ficheros de código.

- Carpeta data:

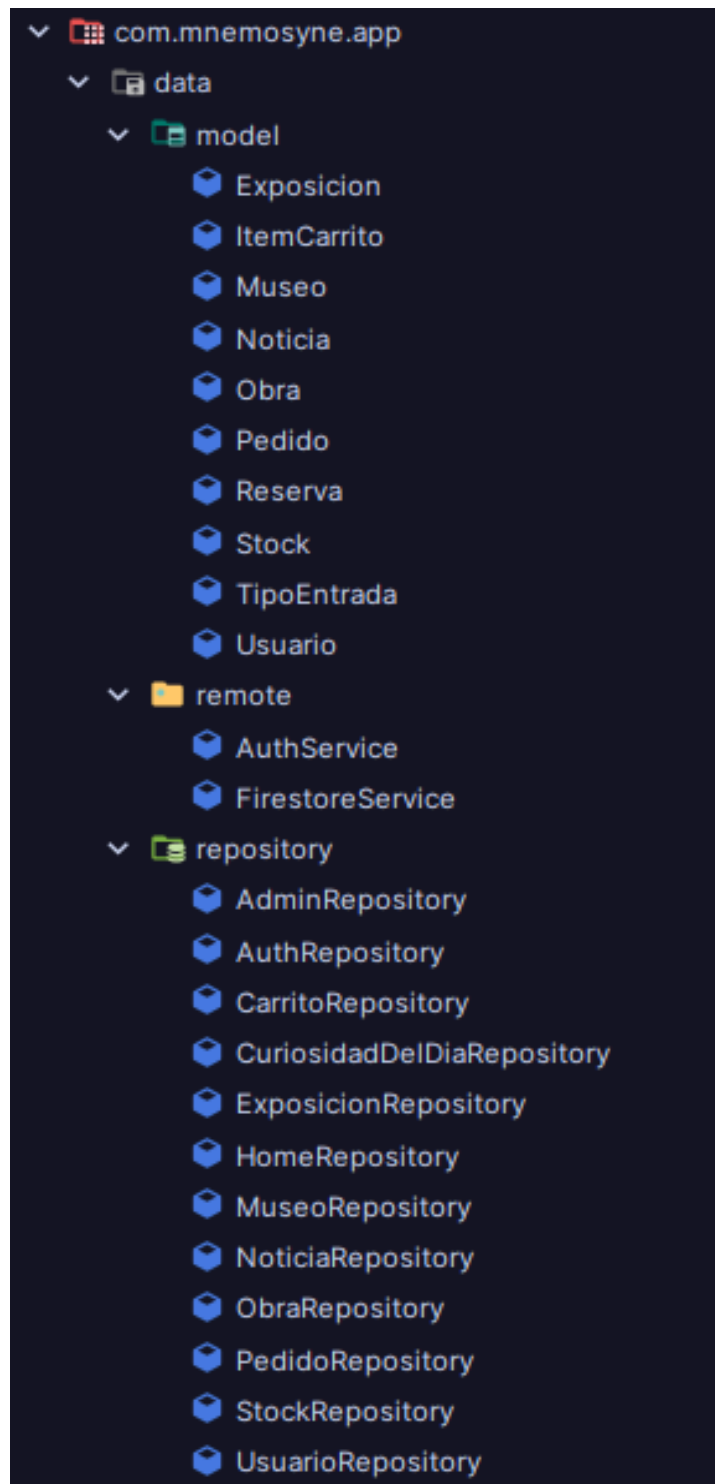


Figura 4. Estructura de paquetes de la capa de datos de la aplicación Mnemosyne.

- Carpeta UI:

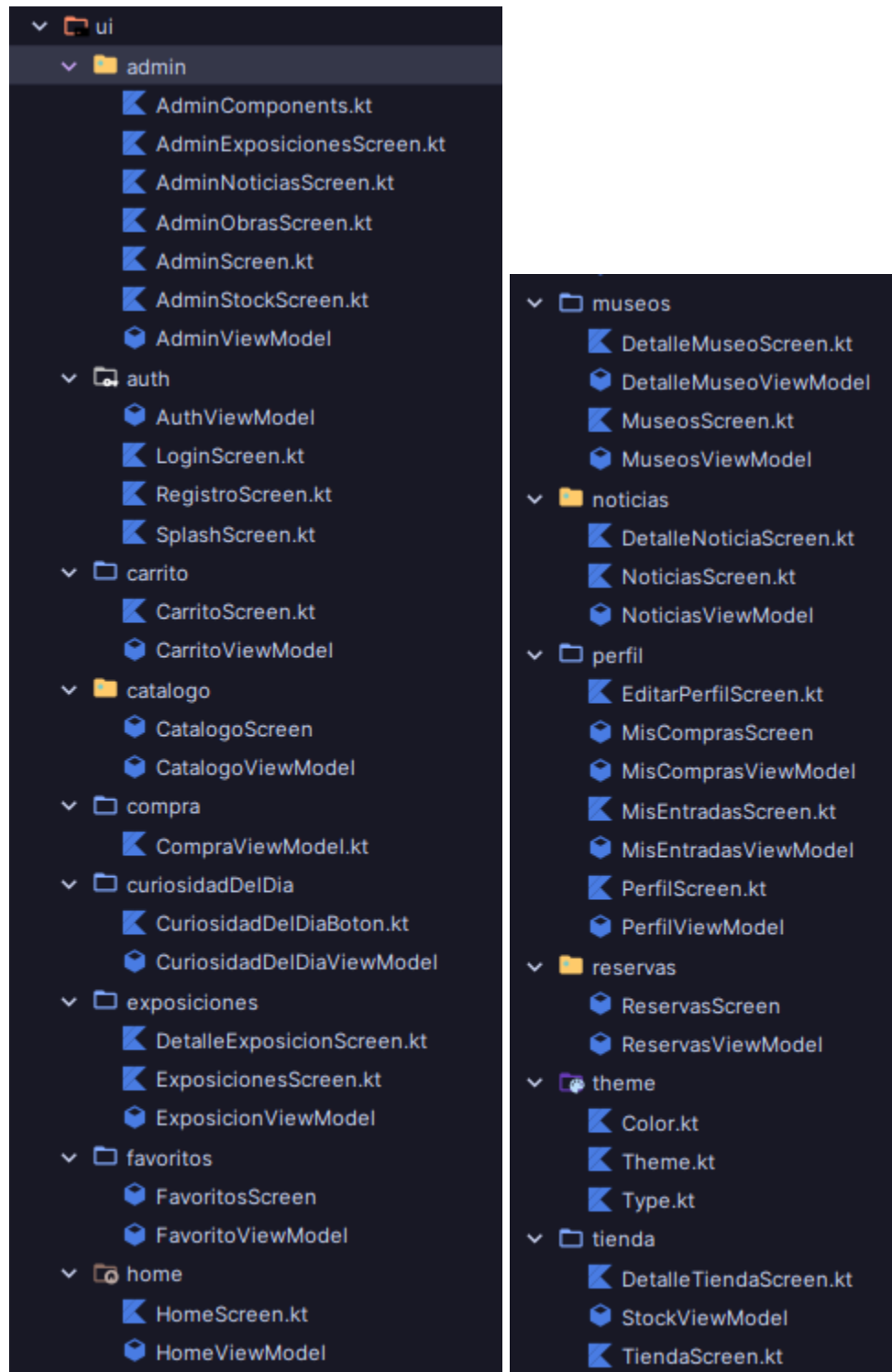


Figura 5. Estructura de paquetes de la capa de interfaz de usuario (UI) de la aplicación Mnemosyne.

- Carpeta utils:

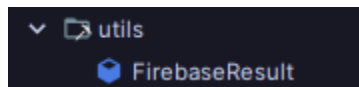


Figura 6. Estructura del paquete utils de la aplicación Mnemosyne.

Diseño/diagramas de clases.

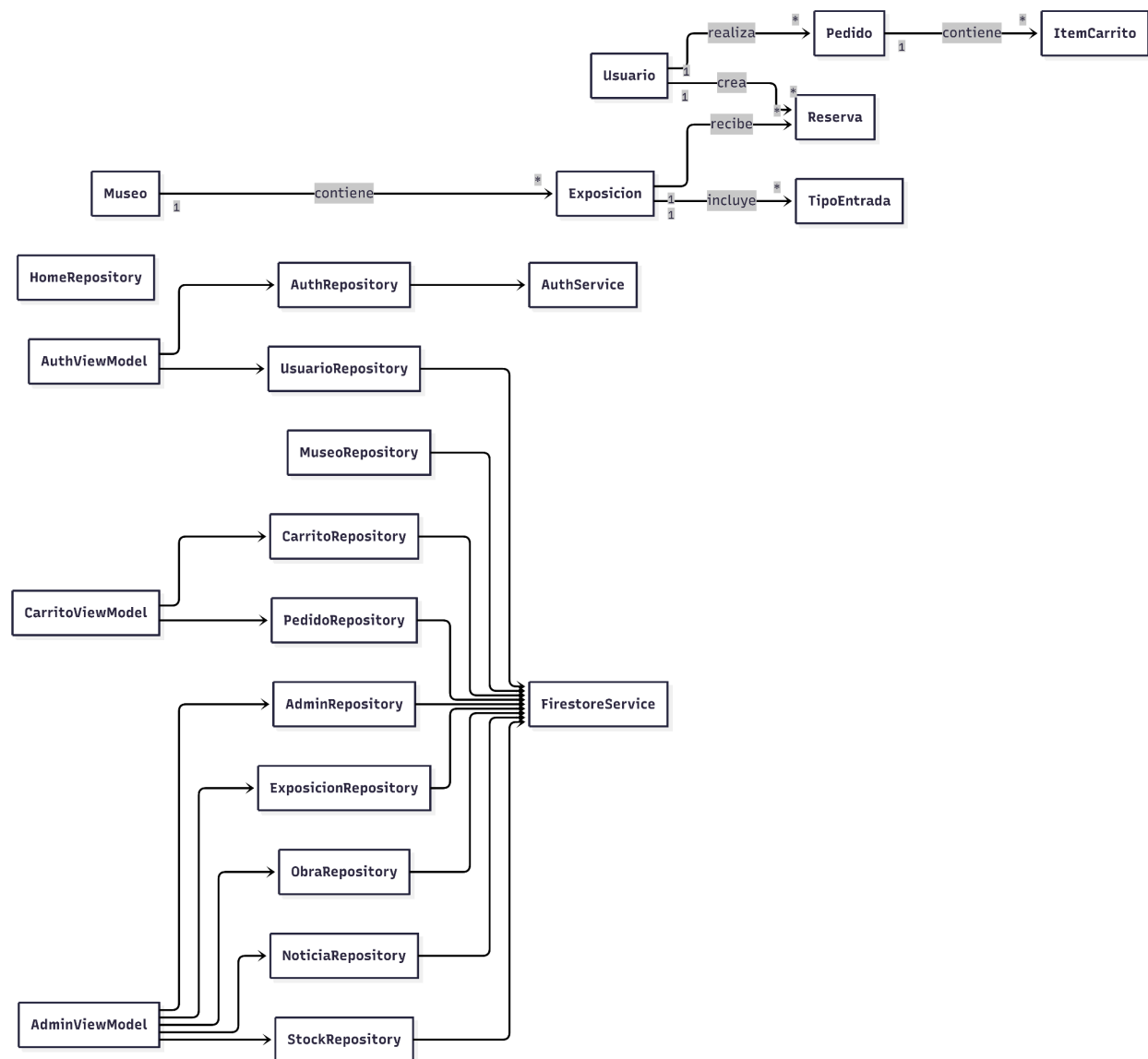


Figura 7. Diagrama de arquitectura y relaciones entre componentes de la aplicación Mnemosyne.

Diagrama de navegación.

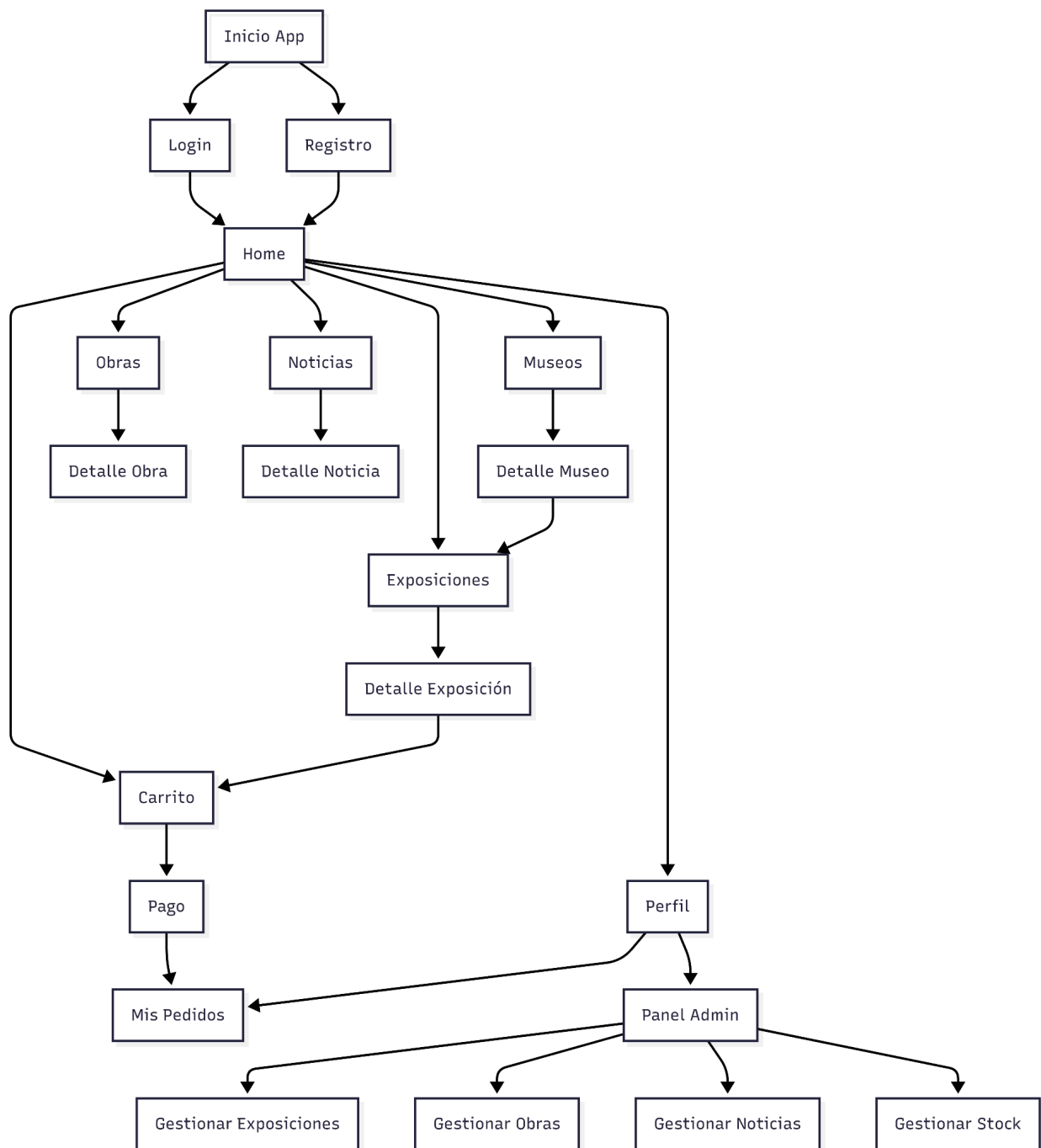


Figura 8. Diagrama de navegación de la aplicación Mnemosyne.

VI. Trabajo de Implementación.

VI.I - Introducción:

La aplicación desarrollada consiste en una aplicación Android orientada a la gestión y difusión de contenido cultural y museístico. La plataforma permite a los usuarios consultar museos, exposiciones, noticias y productos de tienda, además de adquirir entradas y gestionar compras desde un único entorno digital.

La aplicación ofrece diferentes funcionalidades dependiendo del rol del usuario.

Los usuarios estándar pueden registrarse, iniciar sesión, consultar información cultural, comprar entradas y productos, así como gestionar su perfil y pedidos.

Por otro lado, los administradores disponen de un panel de administración desde el que pueden gestionar exposiciones, noticias, obras y productos de tienda.

El proyecto se ha desarrollado siguiendo una arquitectura moderna basada en Firebase y Kotlin, utilizando servicios cloud para almacenamiento, autenticación y persistencia de datos en tiempo real.

VI.II - Tecnologías y Herramientas

Para el desarrollo de la aplicación se ha utilizado una arquitectura móvil basada en Android Studio y Firebase, empleando las siguientes tecnologías.

Desarrollo móvil:

- Kotlin: lenguaje utilizado para el desarrollo de la aplicación Android.
- Android Studio: entorno de desarrollo integrado (IDE) utilizado para la implementación del proyecto.
- Jetpack Compose: framework moderno de Android utilizado para la construcción de interfaces declarativas.
- Corrutinas de Kotlin: utilizadas para gestionar operaciones asíncronas sin bloquear la interfaz del usuario.
- Navigation Compose: librería utilizada para la navegación entre pantallas.

Backend y almacenamiento:

- Firestore Authentication: servicio utilizado para el registro e inicio de sesión de usuarios.
- Cloud Firestore: base de datos NoSQL utilizada para almacenar usuarios, museos, exposiciones, noticias, pedidos y productos.
- Cloudinary: servicio externo utilizado para el almacenamiento y gestión de imágenes mediante URLs.

Librerías y herramientas adicionales:

- Coil: librería utilizada para la carga asíncrona de imágenes desde URLs externas.
- Stripe: plataforma de pago integrada para procesar compras de entradas y productos.
- GitHub: plataforma utilizada para el control de versiones y almacenamiento del código fuente.

VI.III - Desarrollo**Arquitectura de la aplicación.**

La aplicación se desarrolló siguiendo una arquitectura basada en repositorios, separando claramente la lógica de acceso a datos de la interfaz de usuario. Cada módulo funcional dispone de su propio repositorio encargado de comunicarse con Firestore y gestionar las operaciones CRUD correspondientes.

Entre los principales repositorios destacan:

- HomeRepository
- MuseoRepository
- ExposicionRepository
- NoticiaRepository
- CarritoRepository
- PedidoRepository
- UsuarioRepository

- AdminRepository
- StockRepository

Esta estructura facilita el mantenimiento del código, mejora la escalabilidad y favorece la reutilización de lógica dentro de la aplicación.

Implementación del sistema de autenticación.

El sistema de autenticación se implementó mediante Firebase Authentication, permitiendo el registro e inicio de sesión mediante correo electrónico y contraseña. Además, la aplicación valida automáticamente la existencia del documento del usuario en Firestore. En caso de no existir, este se crea automáticamente utilizando el UID proporcionado por Firebase Authentication.

El sistema valida previamente los campos introducidos por el usuario antes de realizar la petición a Firebase Authentication. Posteriormente, se utilizan corrutinas de Kotlin (viewModelScope.launch) para ejecutar la autenticación de forma asíncrona evitando bloquear la interfaz de usuario.

También se implementó un sistema de roles mediante el campo esAdmin almacenado en Firestore. Cuando este campo es verdadero, el usuario obtiene acceso al panel de administración.



Figura 9.

Pantalla de inicio de sesión.

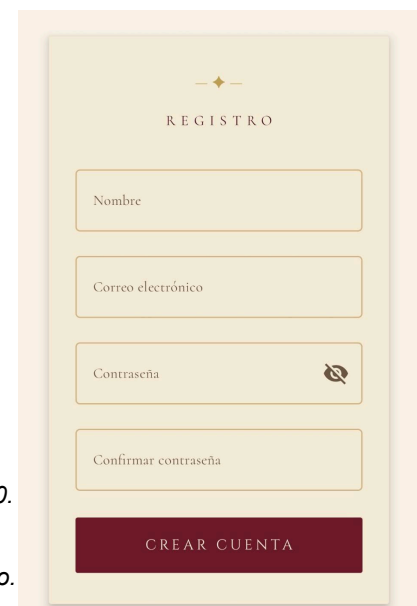


Figura 10.

Pantalla de registro.

La pantalla de inicio de sesión presenta dos campos de entrada: correo electrónico y contraseña, junto con un botón de acceso y un enlace directo al registro. La pantalla de registro solicita nombre, correo electrónico, contraseña y confirmación de contraseña. En ambos casos, el sistema valida los campos antes de realizar cualquier petición a Firebase Authentication.

```
fun login(email: String, password: String) {  
    when {  
        email.isBlank() && password.isBlank() -> {  
            _loginState.value = FirebaseResult.Error( mensaje = "Rellena el correo y la contraseña")  
            return  
        }  
        email.isBlank() -> {  
            _loginState.value = FirebaseResult.Error( mensaje = "Introduce tu correo electrónico")  
            return  
        }  
        password.isBlank() -> {  
            _loginState.value = FirebaseResult.Error( mensaje = "Introduce tu contraseña")  
            return  
        }  
    }  
    _loginState.value = FirebaseResult.Loading  
    viewModelScope.launch {  
        _loginState.value = repository.login(email, password)  
    }  
}
```

Figura 11. Fragmento del AuthViewModel: validación de campos en el método login().

El método login() comprueba mediante un bloque when que los campos de correo electrónico y contraseña no estén vacíos antes de realizar la petición a Firebase Authentication. En caso de que alguno esté en blanco, se actualiza el estado con un mensaje de error específico sin llegar a contactar con el servidor.

```
fun registro(nombre: String, email: String, password: String) {  
    if (nombre.isBlank() || !validarCampos(email, password)) {  
        _registroState.value = FirebaseResult.Error(mensaje = "Rellene todos los campos")  
        return  
    }  
    _registroState.value = FirebaseResult.Loading  
    viewModelScope.launch {  
        _registroState.value = repository.registro(nombre, email, password)  
    }  
}
```

Figura 12. Fragmento del AuthViewModel: validación de campos en el método registro().

El método registro() verifica que el nombre y el resto de campos no estén vacíos antes de proceder. Una vez superada la validación, se lanza una corrutina con viewModelScope.launch que ejecuta el registro en Firebase Authentication de forma asíncrona, actualizando el estado mediante LiveData con el resultado obtenido.

Gestión de datos con Firestore.

Cloud Firestore se utilizó como base de datos principal de la aplicación debido a su integración con Firebase y su capacidad de sincronización en tiempo real.

La estructura de datos se organizó mediante colecciones y subcolecciones jerárquicas asociadas a cada museo. Entre las principales entidades implementadas destacan:

- Museos.
- Exposiciones.
- Noticias.
- Obras.
- Stock.

Además, se implementaron colecciones independientes para la gestión global de la aplicación:

- Usuarios.
- Carritos.

- Pedidos.

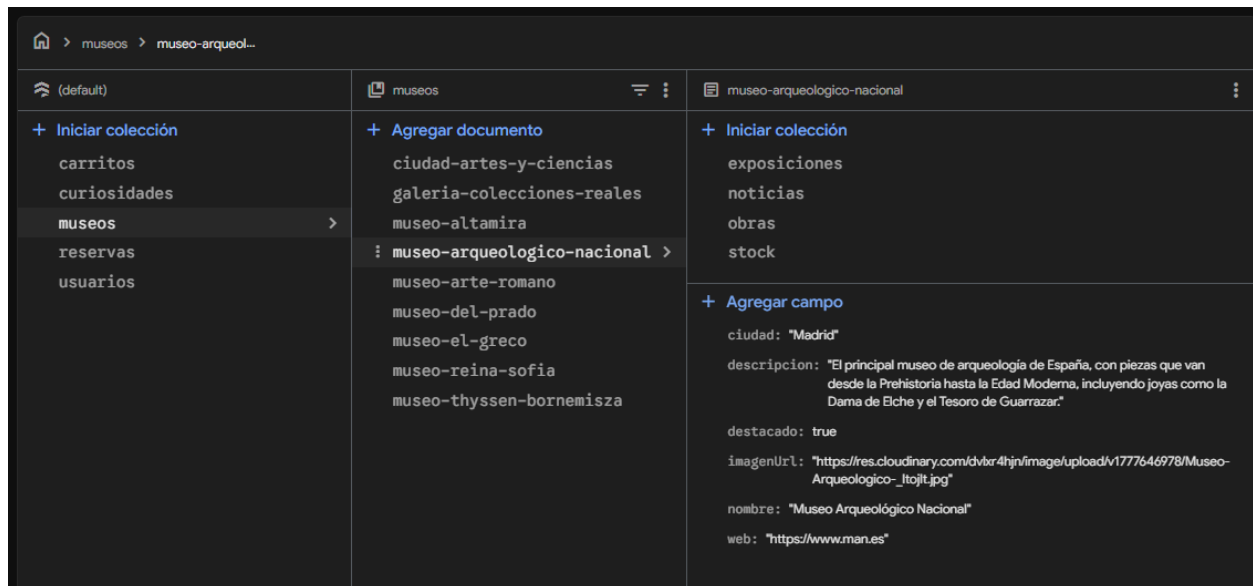


Figura 13. Estructura jerárquica de colecciones y subcolecciones implementada en Cloud Firestore.

La utilización de Firestore permitió actualizar dinámicamente la información mostrada en la aplicación sin necesidad de publicar nuevas versiones. Asimismo, la organización jerárquica de los datos facilitó la escalabilidad del sistema y simplificó la gestión independiente del contenido asociado a cada museo.

Implementación del carrito y sistema de compras.

El sistema de carrito se implementó utilizando Firestore como almacenamiento persistente. Cada usuario dispone de un carrito propio asociado a su UID, permitiendo mantener los productos seleccionados entre sesiones y dispositivos.

Las principales funcionalidades implementadas fueron:

- **Añadir obras o productos al carrito**, permitiendo al usuario seleccionar los elementos deseados desde el catálogo. Cada elemento añadido incluye información básica como identificador, título, precio y cantidad.

```
suspend fun añadirItem(item: ItemCarrito): FirebaseResult<Unit> {  
    return try {  
        val data = mapOf(  
            "exposicionId"      to item.exposicionId,  
            "exposicionTitulo"  to item.exposicionTitulo,  
            "museoId"           to item.museoId,  
            "museoNombre"       to item.museoNombre,  
            "productoId"        to item.productoId,  
            "tipoEntradaNombre"  to item.tipoEntradaNombre,  
            "precio"             to item.precio,  
            "cantidad"          to item.cantidad,  
            "categoria"         to item.categoria  
        )  
        itemsRef().add(data).await()  
        FirebaseResult.Success( data = Unit)  
    } catch (e: Exception) {  
        FirebaseResult.Error( mensaje = e.message ?: "Error al añadir al carrito")  
    }  
}
```

Figura 14. Implementación del método `añadirItem()` en el `CarritoRepository`.

- **Actualización de cantidades**, de forma que el usuario puede incrementar o reducir el número de unidades de un mismo producto dentro del carrito, recalculándose automáticamente el importe total.

```
suspend fun actualizarCantidad(itemId: String, cantidad: Int): FirebaseResult<Unit> {  
    return try {  
        itemsRef().document( documentPath = itemId).update( field = "cantidad", value = cantidad).await()  
        FirebaseResult.Success( data = Unit)  
    } catch (e: Exception) {  
        FirebaseResult.Error( mensaje = e.message ?: "Error al actualizar cantidad")  
    }  
}
```

Figura 15. Implementación del método `actualizarCantidad()` en el `CarritoRepository`.

- **Eliminación de elementos**, permitiendo retirar productos individualmente del carrito sin afectar al resto de artículos seleccionados.

```

suspend fun eliminarItem(itemId: String): FirebaseResult<Unit> {
    return try {
        itemsRef().document( documentPath = itemId).delete().await()
        FirebaseResult.Success( data = Unit)
    } catch (e: Exception) {
        FirebaseResult.Error( mensaje = e.message ?: "Error al eliminar del carrito")
    }
}

```

Figura 16. Implementación del método `eliminarItem()` en el `CarritoRepository`.

- **Cálculo dinámico del total**, basado en la suma de los precios de los productos multiplicados por sus respectivas cantidades, garantizando una actualización en tiempo real.

```

fun calcularTotal(items: List<ItemCarrito>): Double {
    return items.sumOf { it.precio * it.cantidad }
}

```

Figura 17. Implementación del cálculo del total del carrito mediante la suma de precios y cantidades.

- **Persistencia del carrito en Firestore**, lo que permite que los datos del carrito se mantengan almacenados incluso tras cerrar la sesión o cambiar de dispositivo, gracias a la asociación con el identificador único del usuario (UID).
- **Sincronización en tiempo real**, aprovechando las capacidades de Firestore para reflejar inmediatamente cualquier cambio realizado en el carrito en la interfaz de usuario.

Integración Continua y Despliegue (CI/CD)

Para la gestión del ciclo de desarrollo se utilizaron dos herramientas principales que permitieron mantener el control del código fuente y garantizar la coherencia entre el entorno local y el entorno de producción.

El control de versiones se gestionó mediante **GitHub**, organizando el desarrollo en ramas y registrando el historial de cambios de forma incremental a lo largo de todo el proyecto. Esta práctica facilitó la trazabilidad de las modificaciones realizadas en cada módulo y permitió revertir cambios en caso de errores durante el desarrollo.

Para el despliegue de la lógica de servidor, se utilizó la **Firebase CLI**, que permitió publicar las reglas de seguridad de Firestore y las Cloud Functions directamente desde el entorno local. Esto garantizó que las reglas activas en producción estuvieran siempre sincronizadas con la versión estable del código, evitando inconsistencias entre el comportamiento esperado y el entorno real.

VI.IV. Justificación

La elección de las tecnologías y herramientas utilizadas en el desarrollo de Mnemosyne se basó en los siguientes criterios:

- **Robustez y Seguridad:** Firebase Authentication proporciona un sistema de autenticación seguro y ampliamente probado, eliminando la necesidad de gestionar credenciales de forma manual. Las Cloud Functions permiten ejecutar la lógica crítica del proceso de pago en un entorno seguro y aislado, protegiendo la integridad de las transacciones realizadas a través de Stripe.
- **Escalabilidad y Rendimiento:** La arquitectura Serverless de Firebase permite que la aplicación escale automáticamente en función de la demanda, sin necesidad de gestionar infraestructura de servidor. Cloud Firestore garantiza la sincronización de datos en tiempo real con una latencia mínima, mientras que Cloudinary optimiza la entrega de contenido multimedia mediante carga progresiva.
- **Facilidad de Desarrollo:** Kotlin y Jetpack Compose ofrecen un entorno de desarrollo moderno y expresivo para Android, reduciendo la cantidad de código necesario y mejorando la legibilidad. La integración nativa de Firebase con Android Studio simplificó la configuración del proyecto y aceleró el desarrollo de las funcionalidades principales.
- **Integración Continua y Despliegue:** GitHub permitió mantener un control exhaustivo del historial de cambios durante todo el desarrollo, mientras que la Firebase CLI facilitó el despliegue automatizado de reglas de seguridad y funciones de nube, garantizando la coherencia entre el entorno local y el entorno de producción.

VIII - Trabajo de Pruebas.

VIII.I - Introducción.

Se han realizado pruebas funcionales manuales sobre la aplicación Mnemosyne con el objetivo de verificar el correcto comportamiento de las funcionalidades implementadas y validar la integración entre la interfaz de usuario, la lógica de negocio y los servicios cloud utilizados (Firebase Authentication, Cloud Firestore y Stripe).

Las pruebas se ejecutaron en dispositivos Android y emuladores desde Android Studio, simulando diferentes perfiles de usuario y escenarios de uso tanto para usuarios estándar como para administradores.

VIII.II - Pruebas del sistema de autenticación.

Registro de usuarios con datos válidos:

Se comprobó que el sistema permite registrar nuevos usuarios correctamente mediante Firebase Authentication.

Flujo probado:

- Introducción de nombre, correo y contraseña válidos.
- Creación de cuenta.
- Generación automática del documento del usuario en Firestore.

Resultado obtenido:

- Registro completado correctamente.
- Usuario almacenado en Firebase Authentication.
- Documento creado en la colección usuarios.

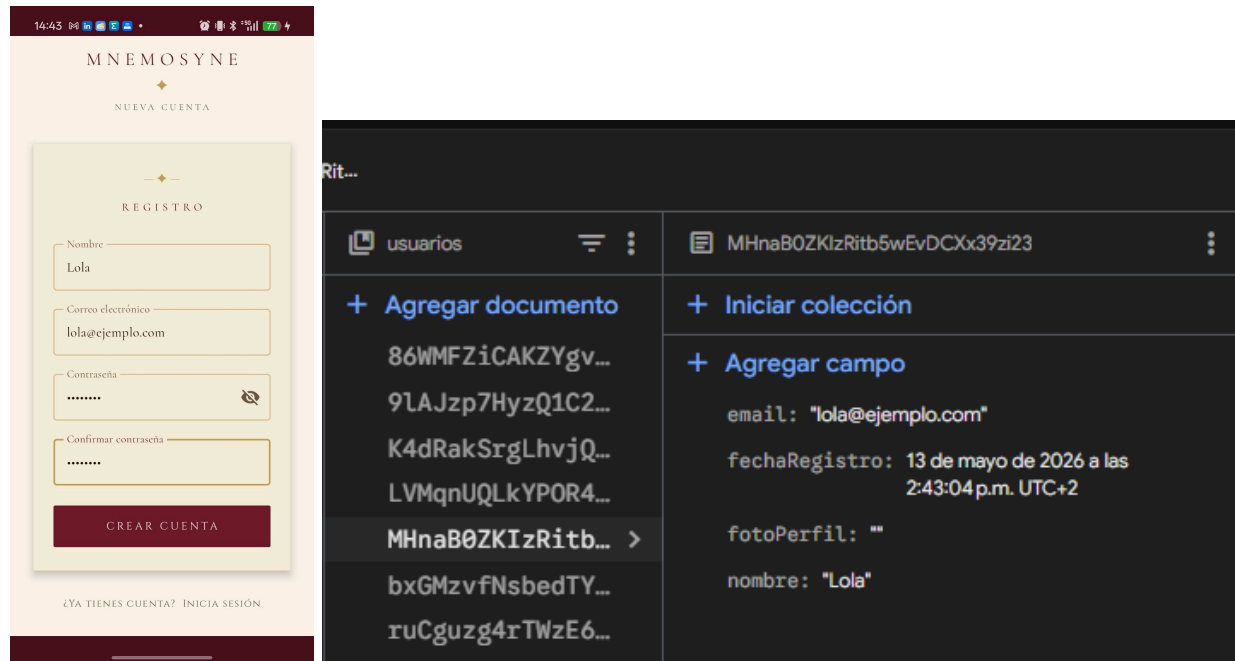


Figura 18. Pantalla de registro completado correctamente.

Validación de campos vacíos:

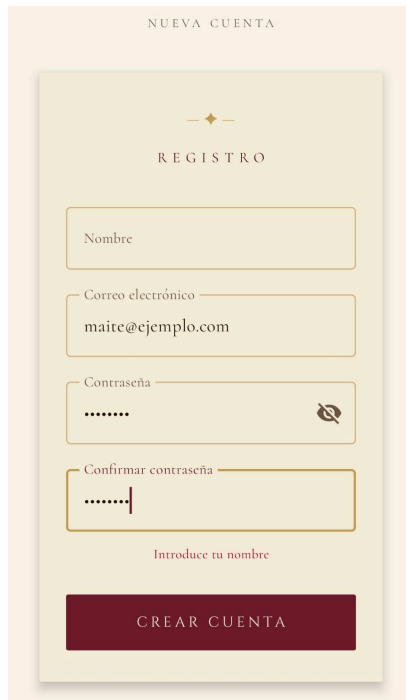
Se verificó que la aplicación impide continuar el proceso de autenticación cuando existen campos obligatorios vacíos.

Casos probados:

- Correo vacío.
- Contraseña vacía.
- Nombre vacío en el registro.

Resultado obtenido:

- El sistema muestra mensajes de error personalizados en español sin realizar peticiones a Firebase.



NUEVA CUENTA

REGISTRO

Nombre

Correo electrónico
maite@ejemplo.com

Contraseña
.....

Confirmar contraseña
.....

Introduce tu nombre

CREAR CUENTA

The registration form is titled 'NUEVA CUENTA' and 'REGISTRO'. It contains four input fields: 'Nombre', 'Correo electrónico' (with the value 'maite@ejemplo.com'), 'Contraseña' (with masked characters '.....'), and 'Confirmar contraseña' (with masked characters '.....'). Below the password fields is a red text message 'Introduce tu nombre'. At the bottom is a dark red button labeled 'CREAR CUENTA'.

Figura 19. Validación de campos obligatorios vacíos.

Inicio de sesión con credenciales incorrectas:



Inicio de sesión

Correo electrónico
heyeusje@jejdj.com

Contraseña
.....

Correo o contraseña incorrectos

ENTRAR

¿NO TIENES CUENTA? REGÍSTRATE

The login form is titled 'Inicio de sesión'. It contains two input fields: 'Correo electrónico' (with the value 'heyeusje@jejdj.com') and 'Contraseña' (with masked characters '.....'). Below the password field is a red text message 'Correo o contraseña incorrectos'. At the bottom is a dark red button labeled 'ENTRAR'. At the very bottom, there is a link '¿NO TIENES CUENTA? REGÍSTRATE'.

Figura 20. Validación de credenciales.

Inicio de sesión con credenciales correctas:



Figura 21. Inicio de sesión válido.

Se ha verificado que, tras el registro, se crea correctamente el documento del usuario en Cloud Firestore asociado a su UID.

VIII.III - Pruebas de Integración y Sincronización Real-Time.

Esta fase validó la comunicación entre la aplicación y el ecosistema **Firebase/Stripe**:

Persistencia Firestore - Se verificó que tras el registro manual de un usuario, el documento correspondiente en la colección usuarios se creará automáticamente con el UID correcto y los campos inicializados.

Flujo de Pagos (Stripe) - Se realizaron simulaciones de compra utilizando el entorno de pruebas de **Stripe**. Se comprobó que el pedido se registraba en la subcolección pedidos del usuario solo tras recibir la confirmación de pago exitosa.

Gestión Multimedia - Verificación del almacenamiento híbrido comprobando que las URLs generadas por **Cloudinary** se cargaban de forma progresiva en la interfaz mediante la librería Coil.

VIII.IV - Pruebas Funcionales y Gestión de Errores.

Campos Vacíos - El sistema detecta mediante un bloque when si el correo o la contraseña están en blanco, mostrando mensajes de error personalizados.

Seguridad de Roles - Se validó que solo los usuarios con el campo esAdmin = true en Firestore pueden visualizar y acceder al panel de administración.

Navegación Reactiva - Se comprobó que al eliminar un ítem del carrito, el importe total se recalcula instantáneamente en la interfaz.

```
class ExampleUnitTest {
    @Test
    fun carrito_calculoTotal_esCorrecto() {
        val precioEntrada = 3.00
        val cantidad = 3

        val totalEsperado = 9.00
        val totalReal = precioEntrada * cantidad

        assertEquals( expected = totalEsperado, actual = totalReal, delta = 0.001)
    }

    @Test
    fun registro_camposVacios_retornaError() {
        val nombre = ""
        val email = "test@ejemplo.com"

        val esValido = nombre.isNotEmpty() && email.isNotEmpty()

        assertFalse( message = "El registro no debería ser válido con nombre vacío", condition = esValido)
    }

    @Test
    fun producto_sinStock_deshabilitaCompra() {
        val cantidadDisponible = 0

        val esComprable = cantidadDisponible > 0

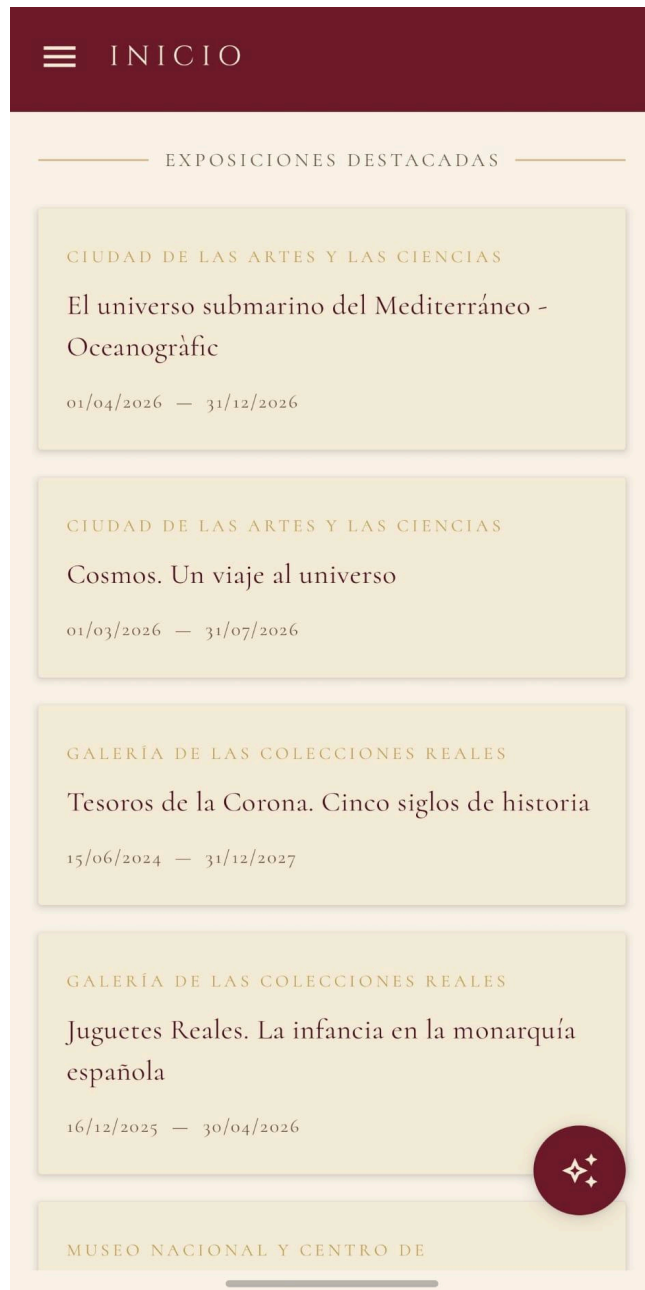
        assertFalse( message = "El producto no debería permitirse en el carrito sin stock", condition = esComprable)
    }
}
```

Figura 22. Tests realizados en JUnit.

SECCIÓN III. PRODUCTO

X. Descripción externa

Pantalla de Inicio:



La pantalla de inicio actúa como el punto central de navegación de la aplicación. En ella se muestran exposiciones y noticias destacadas obtenidas dinámicamente desde Cloud Firestore. Además, incorpora accesos directos a las principales secciones de la aplicación: exposiciones, museos, noticias, tienda, carrito y perfil de usuario.

La interfaz ha sido diseñada utilizando Jetpack Compose, ofreciendo una experiencia moderna, fluida e intuitiva. El contenido se carga de forma asíncrona mediante corrutinas de Kotlin, evitando bloqueos en la interfaz y garantizando una navegación rápida.

Registro de usuarios:



La pantalla de registro permite crear nuevas cuentas mediante Firebase Authentication. El formulario solicita nombre, correo electrónico y contraseña, aplicando validaciones previas para evitar campos vacíos o datos incorrectos.

Tras completar el registro correctamente, el sistema crea automáticamente el documento correspondiente del usuario en Cloud Firestore utilizando el UID generado por Firebase Authentication.

Inicio de sesión:



La interfaz de inicio de sesión permite a los usuarios autenticarse mediante correo electrónico y contraseña. El sistema valida previamente los campos introducidos y muestra mensajes de error personalizados cuando las credenciales son incorrectas o existen campos vacíos.

Una vez autenticado, el usuario obtiene acceso a las funcionalidades disponibles según su rol dentro de la aplicación.

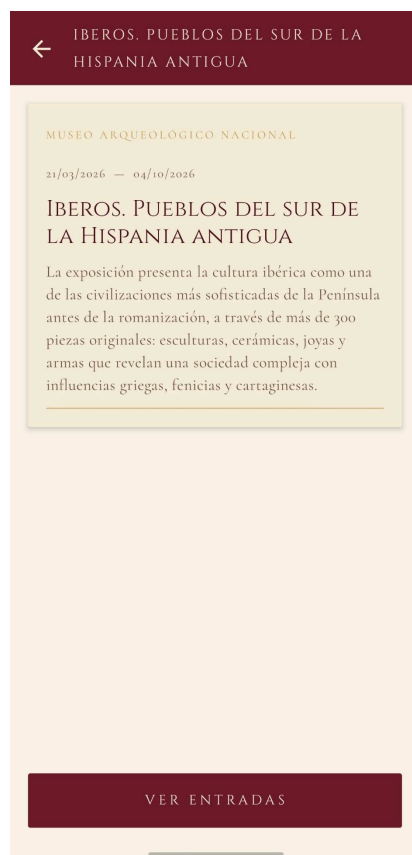
Exposiciones:



La sección de exposiciones muestra todas las exposiciones disponibles asociadas a los distintos museos. El usuario puede filtrar exposiciones por museo mediante chips interactivos y acceder al detalle de cada una.

Cada exposición incluye:

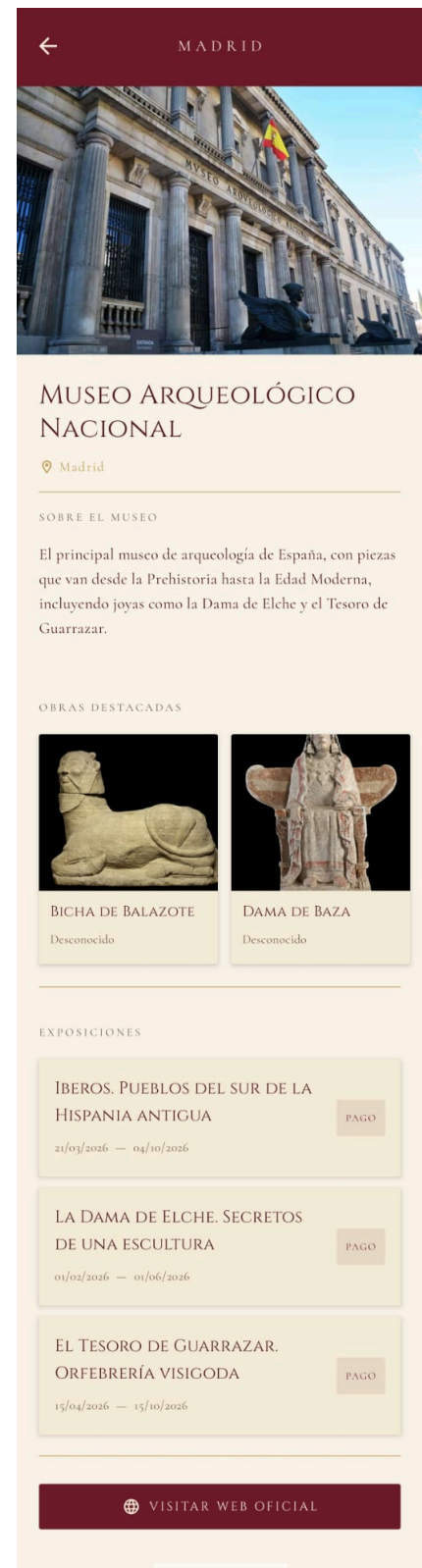
- Descripción completa.
- Fechas de inicio y finalización.
- Tipos de entradas disponibles.
- Posibilidad de añadir entradas al carrito.



Museos:



La pantalla de museos presenta el catálogo completo de museos integrados en la aplicación. Incluye una barra de búsqueda que permite filtrar por nombre o ciudad en tiempo real.

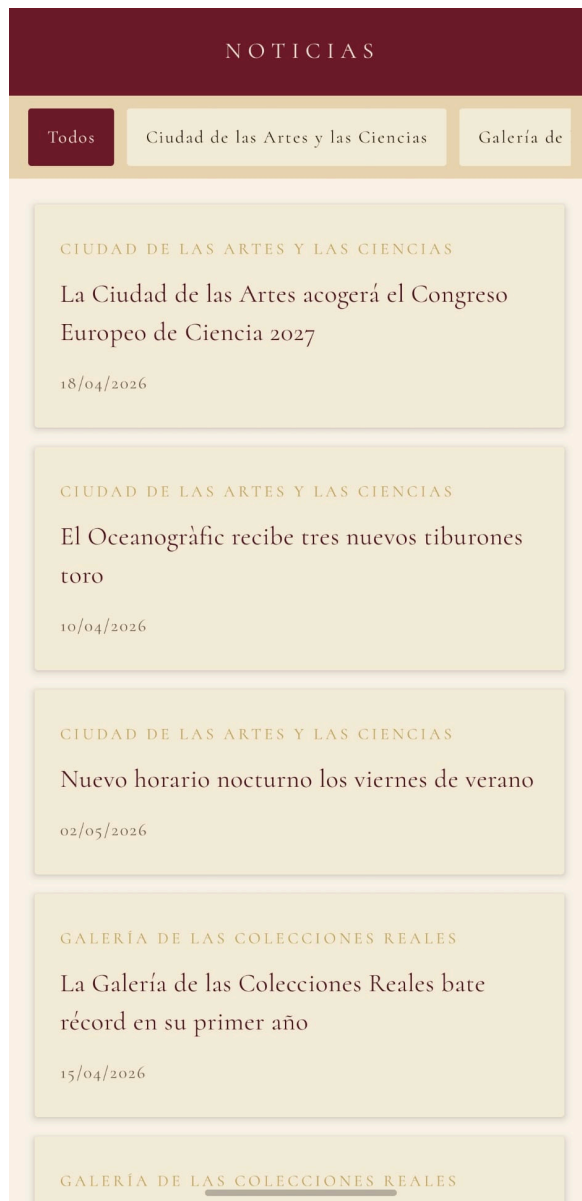


Cada museo muestra información básica como:

- Nombre.
- Ciudad.
- Imagen.
- Descripción resumida.
- Indicador de museo destacado.

Desde el detalle del museo, el usuario puede acceder directamente a la página web oficial del centro cultural.

Noticias:



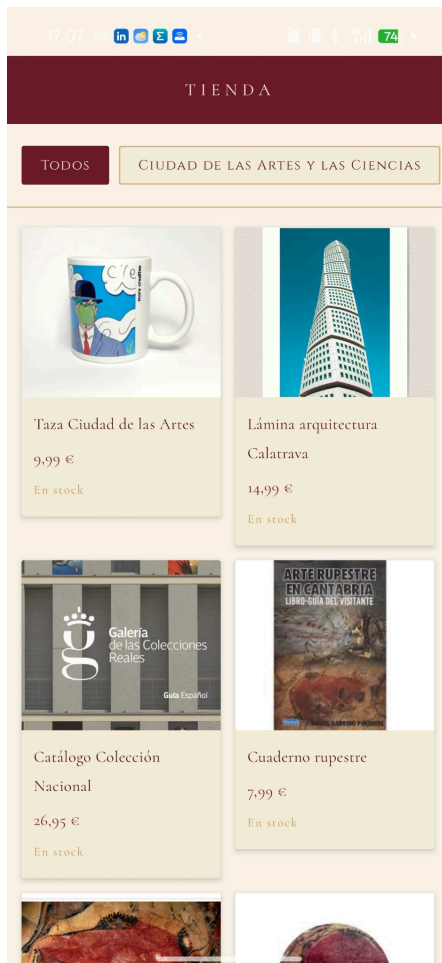
La sección de noticias centraliza las publicaciones culturales procedentes de todos los museos registrados en la plataforma.

Las noticias pueden filtrarse por museo y muestran:

- Título.
- Fecha de publicación.
- Museo de origen.
- Contenido completo de la noticia.



Tienda:



La tienda muestra los productos disponibles asociados a los museos colaboradores. Los productos aparecen organizados en formato de cuadrícula para facilitar la navegación visual.

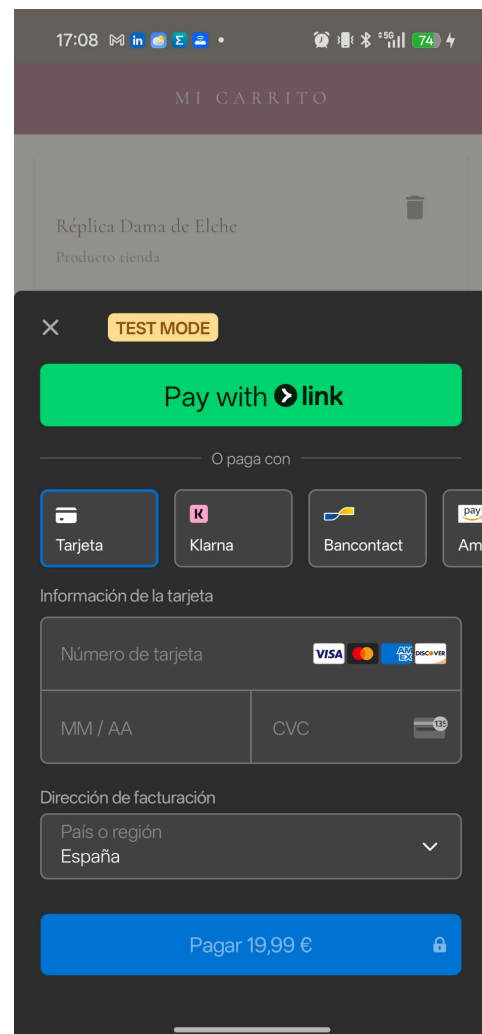
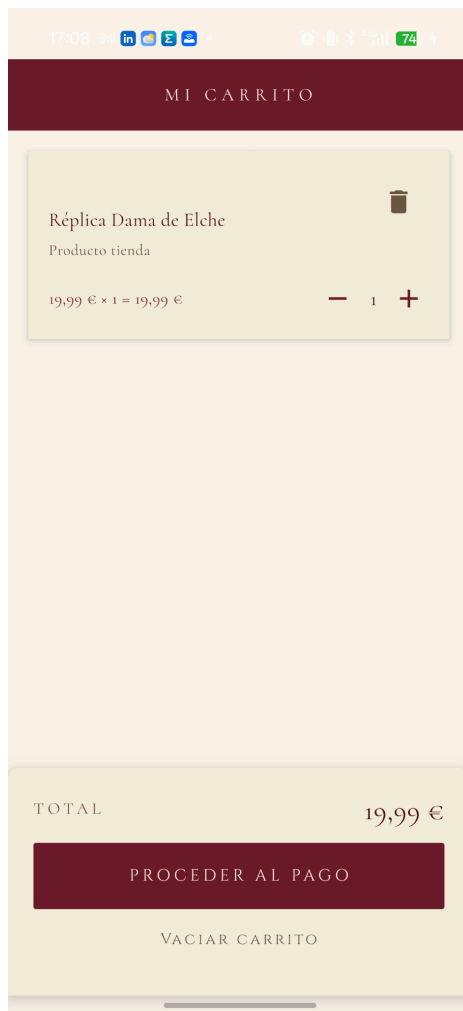
Cada producto incluye:

- Imagen.
- Nombre.
- Precio.
- Disponibilidad de stock.

El usuario puede acceder al detalle del producto y añadirlo al carrito si existe stock disponible.



Carrito y simulación de compra:



El carrito almacena los productos y entradas seleccionados por el usuario utilizando Firestore como almacenamiento persistente.

Desde esta sección el usuario puede:

- Añadir o eliminar productos.
- Modificar cantidades.
- Consultar el importe total.
- Finalizar el proceso de compra.



La integración con Stripe permite simular y procesar pagos de forma segura.

Perfil y pedidos:



La pantalla de perfil permite consultar y modificar la información básica del usuario autenticado, incluyendo:

- Nombre.
- Correo electrónico.
- Foto de perfil.
- Fecha de registro.

Además, el usuario puede consultar el historial de pedidos realizados y las entradas adquiridas, almacenadas en Firestore y ordenadas cronológicamente.



Panel de administración:



Los usuarios con el campo esAdmin = true tienen acceso a un panel de administración exclusivo.

Desde este panel pueden gestionar:

- Exposiciones.
- Noticias.
- Obras.
- Productos de tienda.
- Stock disponible.

Las operaciones CRUD realizadas por el administrador se reflejan automáticamente en la aplicación gracias a la sincronización en tiempo real de Firestore.



XI: Infraestructura:

- La aplicación Mnemosyne se ha desarrollado utilizando tecnologías nativas de Android y servicios cloud de Firebase. Para el desarrollo móvil se emplearon Kotlin, Android Studio y Jetpack Compose, junto con corrutinas y Navigation Compose para la navegación y gestión asíncrona.
- En el backend se utilizaron Firebase Authentication para la gestión de usuarios, Cloud Firestore como base de datos NoSQL y Firebase Cloud Functions para procesos seguros relacionados con pagos. Además, Firebase Storage y Cloudinary se utilizaron para la gestión de imágenes.
- Como herramientas complementarias se integraron Stripe para pagos, GitHub para control de versiones, Firebase CLI para despliegues y Mermaid AI e Instagantt para diagramas y planificación del proyecto.

XII. Mantenimiento

El mantenimiento de Mnemosyne requiere supervisar periódicamente el rendimiento de la aplicación y los servicios de Firebase para detectar posibles errores o problemas de rendimiento.

También es necesario mantener actualizados los contenidos de museos, exposiciones, noticias y productos, además de revisar la consistencia de los datos almacenados en Cloud Firestore.

Por último, deben actualizarse las reglas de seguridad de Firebase y optimizarse las imágenes almacenadas en Cloudinary y Firebase Storage para garantizar seguridad, estabilidad y un buen rendimiento de la aplicación.

SECCIÓN IV. CONCLUSIÓN

XIII. Grado de consecución de objetivos.

El desarrollo de Mnemosyne ha permitido alcanzar de forma satisfactoria la mayoría de los objetivos planteados en el anteproyecto. La aplicación final consigue centralizar en una única plataforma móvil información cultural relacionada con museos, exposiciones, noticias y productos de tienda, ofreciendo una experiencia de usuario unificada y accesible.

Entre los objetivos cumplidos destacan:

- Desarrollo de una aplicación nativa Android utilizando Kotlin y Jetpack Compose.
- Implementación de un sistema de autenticación seguro mediante Firebase Authentication.
- Integración de Cloud Firestore como sistema de persistencia de datos en tiempo real.
- Desarrollo de un sistema de carrito y compras integrado con Stripe.
- Creación de un panel de administración para la gestión de exposiciones, noticias, obras y stock.
- Implementación de una arquitectura MVVM basada en repositorios, facilitando la organización y mantenimiento del código.
- Integración de servicios externos como Cloudinary para la gestión optimizada de contenido multimedia.

Durante el desarrollo fue necesario adaptar parte del alcance inicial definido en el anteproyecto. Algunas funcionalidades planteadas originalmente, como la sincronización multiplataforma completa o ciertos elementos de gamificación, no llegaron a implementarse para priorizar la estabilidad del sistema, la finalización de las funcionalidades principales y la calidad general de la aplicación.

Aun así, el proyecto final cumple los objetivos fundamentales propuestos inicialmente y proporciona una solución funcional, escalable y alineada con las necesidades del sector cultural digital.

XIV. Posible evolución futura del proyecto.

Mnemosyne presenta un gran potencial de ampliación y mejora en futuras versiones. Entre las posibles evoluciones del proyecto destacan:

- Implementación de notificaciones push para informar sobre nuevas exposiciones, eventos o noticias destacadas.
- Desarrollo de un sistema de favoritos y recomendaciones personalizadas según los intereses del usuario.
- Incorporación de funcionalidades de gamificación, como quizzes o recorridos interactivos relacionados con las exposiciones.
- Desarrollo de una versión web o de escritorio sincronizada con la aplicación móvil.
- Implementación de códigos QR para validación digital de entradas.
- Incorporar estadísticas y herramientas analíticas avanzadas en el panel de administración.

Estas posibles mejoras permitirían ampliar las funcionalidades de la plataforma y convertir Mnemosyne en un ecosistema cultural más completo e interactivo.

XV. Lecciones aprendidas.

El desarrollo de Mnemosyne ha supuesto una experiencia práctica muy completa, permitiendo aplicar conocimientos adquiridos durante el ciclo formativo en un proyecto real con una arquitectura moderna y servicios cloud.

Entre las principales lecciones aprendidas destacan:

- La importancia de diseñar una arquitectura bien estructurada para facilitar la escalabilidad y el mantenimiento del proyecto.
- El valor de Firebase como solución Backend as a Service para acelerar el desarrollo y simplificar la gestión de infraestructura.
- La utilidad de las corrutinas de Kotlin para gestionar operaciones asíncronas sin bloquear la interfaz de usuario.
- La importancia de realizar pruebas funcionales continuas para detectar errores y garantizar la estabilidad de la aplicación.

- La necesidad de adaptar el alcance inicial del proyecto a los recursos y tiempos disponibles, priorizando las funcionalidades esenciales.
- La relevancia de una interfaz intuitiva y una experiencia de usuario fluida en aplicaciones móviles.
- La importancia del control de versiones mediante GitHub para mantener la trazabilidad y organización del desarrollo.

En conjunto, el proyecto ha permitido adquirir experiencia tanto en el desarrollo frontend como en la integración de servicios backend, proporcionando una visión global del ciclo completo de desarrollo de una aplicación moderna orientada al entorno profesional.

BIBLIOGRAFÍA

Mermaid AI. (s.f.). *Herramienta para la generación de diagramas y visualización de arquitecturas de software*.

Instagantt. (s.f.). *Software de diagramas de Gantt y gestión de proyectos*.

MoureDev by Brais Moure - Curso de Kotlin desde cero. (2023, junio 16). *Curso de KOTLIN desde cero: Primeros pasos en una hora* [Vídeo]. YouTube.

Android Developers. (s.f.). *Documentación oficial de desarrollo Android*.

Kotlin Documentation. (s.f.). *Documentación oficial del lenguaje Kotlin*.

Jetpack Compose Documentation. (s.f.). *Documentación oficial de Jetpack Compose*.

Firebase Documentation. (s.f.). *Documentación oficial de Firebase*.

Cloud Firestore Documentation. (s.f.). *Documentación oficial de Cloud Firestore*.

Firebase Authentication Documentation. (s.f.). *Documentación oficial de Firebase Authentication*.

Firebase Cloud Functions Documentation. (s.f.). *Documentación oficial de Firebase Cloud Functions*.

Stripe Documentation. (s.f.). *Documentación oficial de integración de pagos con Stripe*.

Cloudinary Documentation. (s.f.). *Documentación oficial de Cloudinary para gestión multimedia*.

Coil Documentation. (s.f.). *Documentación oficial de la librería Coil para carga de imágenes en Android*.

GitHub. (s.f.). *Plataforma de control de versiones y alojamiento de código fuente*.

Android Studio. (s.f.). *Entorno de desarrollo oficial para aplicaciones Android*.

Material Design 3. (s.f.). *Sistema de diseño utilizado como referencia para la interfaz de usuario*.